

SHOWCASE TÉCNICO

Arquitetura & Front-End MAG

Visão técnica abrangente da aplicação fe-traditional-sale

Apresentação completa cobrindo arquitetura React/Vite, fluxos de dados, contratos de API e detalhamento técnico do fluxo de contratação e integração de parceiros.

1. Introdução e Propósito do Projeto

Objetivo da Aplicação

A aplicação **MAG Front-End** (repositório fe-traditional-sale) é uma SPA projetada para viabilizar vendas tradicionais de seguros via parceiros e corretoras, suportando personalização de branding, captura de leads por hash e emissão acelerada de propostas.

Escopo da Apresentação

- **Arquitetura SPA:** Estrutura de navegação e serviços.
- **Fluxo de Dados:** Gestão de estado local, Axios e Storage.
- **Contratos de API:** Modelos de requisição e resposta REST.
- **Mapeamento de Páginas:** Responsabilidade de cada módulo.
- **Deep Dive: Access.jsx:** Entrada, validações e regras de negócio.

Stack Tecnológica Principal

Tecnologia	Função / Utilização
React (Vite)	Framework UI reativo de alta performance
React Router DOM	Gerenciamento de rotas e navegação client-side
Axios	Cliente HTTP com interceptores centralizados
CSS Modules	Estilização escopada com suporte a temas
Session / LocalStorage	Persistência temporária de estado e branding

 **Destaque:** A aplicação conta com um alto nível de flexibilidade, permitindo a personalização dinâmica do branding por parceiro via CNPJ.

2. Arquitetura de Alto Nível

Navegador (Client)

React UI Components
React Router & Hooks
Estado Local & SessionStorage

Camada de Serviços

api.js (Instância Axios)
partnerService / proposalService
partnerBranding (Cache local)

Backend REST APIs

Sales Cap Lead API
Partner Branding API
Proposal Management API

Detalhamento dos Componentes Arquiteturais

Camada de Apresentação (UI)

Construída inteiramente em componentes React funcionais com Hooks. Interface reativa com CSS Modules para isolamento de estilos.

Serviços & Interceptadores

Centralização de chamadas via `src/services/api.js`. Gerencia injetores de cabeçalhos padrão (ex: `cnpj`) e tratamento genérico de exceções.

Comunicação Backend

Integração via endpoints RESTful estruturados. Troca de dados em formato JSON estritamente tipado pelos contratos da API.

3. Fluxo de Dados & Gerenciamento de Estado

Origem	Destino	Meio / Mecanismo	Descrição do Fluxo
UI Form Inputs	Estado Local	USESTATE	Captura dinâmica de inputs do usuário (CNPJ, Hash, IDs do Corretor, SUSEP).
Access.jsx	Storage Navegador	SESSIONSTORAGE	Persiste dados do corretor (@Mag:brokerExternalId, producerId, susep) entre páginas.
Access.jsx	Axios Headers	MEMÓRIA / INSTÂNCIA	Configura o cabeçalho global cnpj para todas as chamadas subsequentes à API.
partnerService	Backend API	HTTP GET	Consulta dados e ativos de branding personalizados do parceiro via CNPJ (/partner/:cnpj).
proposalService	Backend API	HTTP POST / PUT	Cria rascunho de proposta e atualiza dados do cliente durante a navegação do fluxo.
Access.jsx	React Router	NAVIGATE()	Redireciona o usuário para /products ou /proposalflow?id=... conforme os dados.

4. Contratos de API – Lead & Parceiro

4.1 Busca de Lead por Hash

GET /api/sales-cap/v1/lead/:hash

```
{
  "data": {
    "generalInfo": {
      "leadId": "string",
      "partnerVendorId": "string",
      "businessPartnerDocument": "string",
      "offerId": "string",
      "offerCode": "string",
      "totalContribution": 150.00
    },
    "commercialDetails": {
      "vendors": [{
        "mainVendor": true,
        "externalId": "EXP123",
        "producerId": "PROD99",
        "susepId": "SUS1234"
      }]
    },
    "policyHolders": [{
      "personDetails": { "document": "12345678901", "name": "João
Silva" }
    }]
  }
}
```

4.2 Consulta de Parceiro & Branding

GET /api/partner/:cnpj

```
{
  "legalName": "Parceiro Corretora de Seguros LTDA",
  "tradeName": "Parceiro Seguros",
  "document": "12345678000195",
  "branding": {
    "logoUrl": "https://cdn.mag.com.br/logos/partner123.png",
    "colors": {
      "primary": "#0052cc",
      "secondary": "#0747a6"
    }
  }
}
```

 **Cache de Branding:** Os dados de branding retornados são armazenados no localStorage sob a chave @Mag:partnerBranding:\${cnpj} para reuso.

4. Contratos de API – Propostas

4.3 Criação de Proposta

POST /api/proposal

```
/* Payload de Envio */
{
  "productId": "PROD-LIFE-01",
  "offerCode": "OFFER-2026",
  "quantity": 1,
  "totalValue": 150.00,
  "partnerCnpj": "12345678000195",
  "partnerName": "Parceiro Seguros"
}

/* Resposta (201 Created) */
{
  "id": "e8d3a1b2-9f4e-4b3c-8a1d-7e9f2a4b6c8d",
  "status": "draft",
  "createdAt": "2026-08-26T14:00:00Z"
}
```

4.4 Atualização de Proposta

PUT /api/proposal/:id

```
/* Payload de Atualização */
{
  "client_type": "fisica",
  "document_number": "12345678901",
  "form_data": {
    "fullName": "João Silva",
    "email": "joao.silva@email.com",
    "phone": "11999998888"
  }
}

/* Resposta (200 OK) */
{
  "id": "e8d3a1b2-9f4e-4b3c-8a1d-7e9f2a4b6c8d",
  "updated": true
}
```

5. Mapeamento das Páginas (`src/pages/`)

Componente	Rota	Propósito & Funcionalidades Principais	Parâmetros / Estado
Access.jsx	/	Ponto de entrada. Busca de lead por Hash ou login/autenticação manual por CNPJ do parceiro.	hashLead, cnpj, externalId, producerId, susep
Products.jsx	/products	Exibe catálogo de seguros filtrados pelo parceiro selecionado. Permite escolha de plano.	Lê @Mag:brokerExternalId do sessionStorage
ProposalFlow.jsx	/proposalflow	Assistente passo a passo (Wizard) para coleta de dados do cliente, confirmação e geração.	Query String: ?id={proposalId}
Extrato.jsx	/extrato	Exibe o extrato detalhado de transações e parcelamento da proposta finalizada.	Recebe proposalId via Router State
Envio.jsx	/envio	Upload e validação de documentos/ comprovantes vinculados à proposta.	Recebe proposalId do fluxo anterior
ProductsSalesAssistant.jsx	/assistant	Assistente virtual interativo com IA para apoio a vendas e recomendação de produtos.	Integração via API de IA (/ai/suggest)

6. Tela Access.jsx – Estado & Regras de Negócio

Variáveis de Estado do Componente

Estado	Tipo	Finalidade
hashLead	string	Hash do lead digitado para busca automática
cnpj	string	CNPJ do parceiro (formatado / numérico)
externalId	string	Identificador externo do corretor
producerId	string	Identificador do produtor vinculado
susep	string	Registro oficial SUSEP do corretor
isLoading	boolean	Controla exibição de spinners de carregamento
isLookingUpLead	useRef	Evita requisições concorrentes/duplicadas do mesmo hash

Regras de Validação & Exibição

```
// Exibe campos do corretor apenas se CNPJ for válido
const showBrokerFields = cnpj.length >= 14;

// Formulário manual considerado válido
const isFormValid =
  cnpj.length >= 14 &&
  externalId !== "" &&
  susep !== "";
```

- Os campos do corretor são renderizados dinamicamente via `showBrokerFields`.
- O botão **Continuar** permanece desabilitado até que `isFormValid === true` OU um `hashLead` seja preenchido.

6. Tela Access.jsx – Funções de Tratamento

Função: handleLeadLookup()

Executada no evento onBlur do campo Hash ou na submissão:

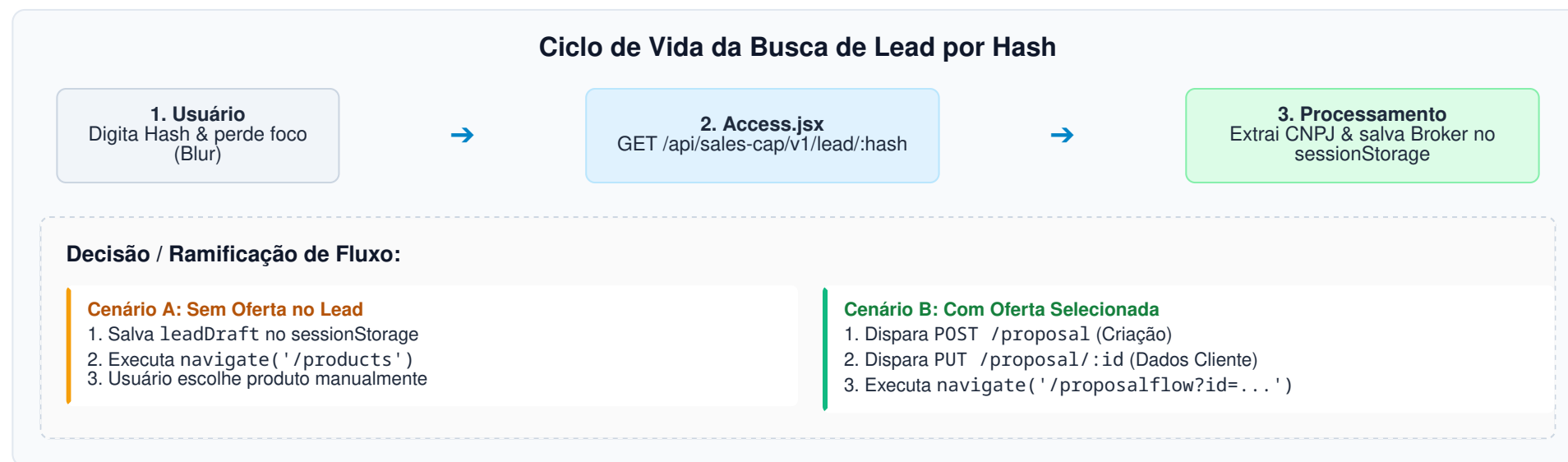
1. Faz trim() no hash e valida se a busca já está em andamento via Ref.
2. Chama GET /api/sales-cap/v1/lead/:hash.
3. Extrai CNPJ do parceiro e armazena credenciais do corretor no sessionStorage.
4. Busca e salva o branding do parceiro via savePartnerBranding.
5. **Ramificação:**
 - Se **NÃO** houver oferta: salva leadDraft e redireciona para /products.
 - Se houver oferta: dispara createProposal + updateProposal e navega para /proposalflow?id=...

Função: handleAccess()

Disparada pelo clique do usuário no botão **Continuar**:

1. **Caminho 1 (Hash Presente):** Delega a execução inteiramente para a função handleLeadLookup().
2. **Caminho 2 (Entrada Manual):**
 - Valida o formulário através da constante isFormValid.
 - Consulta dados de branding com getPartners(cnpj).
 - Persiste os IDs do corretor no sessionStorage.
 - Configura o header padrão cnpj na instância do Axios.
 - Redireciona diretamente para o catálogo em /products.

6. Fluxo de Sequência – Interação da Tela Access



7. Serviços, Rotas e Estilização

Serviços Especializados (`src/services`)

- **api.js:** Instância base do Axios com baseURL e interceptadores globais de requisição e resposta.
- **partnerService.js:** Exporta `getPartners(cnpj)` para busca de dados e ativos visuais do parceiro.
- **partnerBranding.js:** Gerencia o cache local de logos e paleta de cores via `savePartnerBranding()`.
- **proposalService.js:** Abstrai chamadas de criação (`createProposal`) e atualização (`updateProposal`).

Definição de Rotas (`src/App.jsx`)

```
} />  
} />  
} />  
} />  
} />  
} />
```

Estilização & Tematização (CSS Modules)

A aplicação utiliza **CSS Modules** (`*.module.css`) garantindo escopo local de seletores e evitando vazamento de estilos. O tema é ajustável via variáveis CSS dinâmicas injetadas pelo branding do parceiro, com suporte nativo a Dark Mode via consulta de mídia `prefers-color-scheme`.

8. Como Acessar & Checklist de Entrega

Passo a Passo de Execução

1. Iniciar o servidor de desenvolvimento:
`npm run dev`
2. Acessar o navegador em:
`http://localhost:5173/`
3. No componente **Access**, escolha uma opção:
 - Informar um **Hash de Lead** (disparo automático no blur).
 - Preencher o **CNPJ** e dados do corretor para acesso manual.
4. Após validação, navegação automática para `/products` ou `/proposalflow`.

Checklist de Requisitos do Showcase

Item	Status
Diagrama de Arquitetura de Alto Nível	CONCLUÍDO
Tabelas de Contrato de API (Lead, Proposal, Partner)	CONCLUÍDO
Mapeamento Completo de Páginas	CONCLUÍDO
Detalhamento da Tela Access.jsx	CONCLUÍDO
Fluxos de Sequência e Ramificação	CONCLUÍDO